TempCast: A Retrieval-Based Multimodal Framework for Temporal Event Prediction Using Embedding Similarity and Temporal Weighting

Meet Arora, Soumyadipta Ghosh, Yash Sharma
School of Computer Science and Technology
Bennett University, India

***Abstract*—Temporal event prediction in real-world environments remains challenging due to multimodal data heterogenity, temporal irregularities, and noisy observations. Existing statistical and deep learning approaches often fail because they don't generalize effectively across dynamic event distributions while maintaining real-time performance as there always remains discrepancy to some extent. This paper aims to present our solution called TempCast which is a retrieval-based multimodal framework for temporal event prediction. Our proposed system leverages SentenceTransformer and CLIP models for semantic representation, FAISS for efficient similarity search, and temporal decay-based weighting to incorporate temporal dynamics as we think this could fix the issues the previous conventional methods were facing. Unlike conventional sequence models such as LSTM and Transformers, TempCast does it differently as it reformulates predictions as a retrieval problem in embedding space, improving scalability, interpretability, and inference efficiency. Experimental evaluation on a curated multimodal dataset demonstrates improved accuracy and reduced latency compared to baseline approaches and previous solutions.**

***Index Terms*—Temporal Event Prediction, Multimodal Learning, FAISS, Retrieval-Augmented Models, Embedding Similarity**

I. Introduction

Temporal event prediction plays a critical role in situations like disaster management, public safety, and socio-economic forecasting. In countries such as India, frequent occurrences of floods, air pollution crisis, and epidemic outbreaks highlight the need for robust early warning systems as it is very common for disasters to occur frequently due to global warming, rising heat temperatures, health issues and immunity defects in people. Traditional forecasting approaches heavily rely on statistical models or sequential deep learning architectures. While these are effective for structured time-series data, these methods often struggle with multimodal inputs and irregular temporal patterns due to their computational complexity which limits real-time deployment in large-scale systems. Although, recent advancements in artificial intelligence have enabled the use of multimodal data streams, including textual, visual, and temporal signals but existing approaches often suffer from limited interpretability, high computational cost, and insufficient integration of heterogeneous data sources. To address these challenges, we propose our solution TempCast which is a retrieval-based framework that reformulates temporal event prediction as a similarity search problem in embedding space. By leveraging multimodal embeddings and efficient retrieval mechanisms, the system enables scalable and interpretable event forecasting.

II. Related Work

Temporal prediction has been widely studied using classical and deep learning approaches. Statistical models such as ARIMA capture linear temporal dependencies but they fail to model complex real-world dynamics often leading to discrepancies in results Recurrent neural networks, particularly LSTM models [4], improve sequential modeling but they suffer from scalability limitations and long training times often leading to incorrect results. Transformer based architectures address long-range dependencies but introduce significant computational overhead which might lead to overhead expenses in long time run as well. Multimodal learning approaches, such as CLIP [2], enable joint representation learning across text and images. Additionally, retrieval-based methods and nearest-neighbor approaches have been explored in language modeling and recommendation systems. In contrast to these methods, TempCast also integrates multimodal embedding models with FAISS-based retrieval [3] and explicit temporal weighting which provide a scalable and efficient alternative to sequence-based prediction.

III. Dataset and System Design

A. Curated Multimodal Proof-of-Concept Dataset

A major issue we faced early on is that there are hardly any good datasets that map text and images to specific future events. To test our framework, we took the time to build a very clean test dataset of 15 structured event records stored in a simple JSON format. Each entry we created in the dataset contains:

- **Event Description**: A short text summary of what happened (e.g., "Pulwama terror attack kills Indian security personnel in Jammu and Kashmir").
- **Image**: A picture connected to the event.
- **Image Hint**: A short phrase giving background clues about the image added.
- **Event Type**: A category label showing the type of event (e.g., "highlighting").

- **Expected Next Event**: The actual future event that we want the model to predict. (e.g., "India conducts retaliatory airstrikes leading to heightened tensions")
- **Timestamp**: The exact time the event happened.

We organized the dataset this way so the system can look at both the meaning and the timeline of the events. This helps our system learn how events naturally lead into one another, rather than just filling up a bucket.

B. Key Novelty: The Semantic Hint Field

One of the best new ideas we brought to this dataset is the *image hint* field. Most standard systems just feed raw images directly into a neural network. But, we designed the hint field to give a deeper meaning based on what is actually in the picture (e.g., "`military convoy attack, explosion, security forces`").

This works as a helpful guide. It builds a bridge between the picture and the text, which clears up confusion when the system searches for the matches. In cases where a plain picture isn't enough to explain a complicated event (e.g., "A picture of a cyclone cannot determine where it took place and certain other information), the text hint locks down the meaning. It provides rich details that confidently point the search engine toward much more accurate matches of the past.

C. System Interaction Modes

We didn't want to build a confusing "black box" that just types out answers. We designed TempCast as an interactive tool with a few different ways for users to explore it:

- **Forecast Mode**: Guesses the most likely next event based on the current text and picture provided by the user.
- **Retrieval Mode**: Acts as a research tool. It pulls up similar past events so the user can see exactly why the system made its guess.
- **Image Encoding Mode**: Lets users test and look at the image data and text hints on their own.
- **Dataset Exploration Mode**: Gives users a deep look at how the dataset was built and how the events are spaced out over the timeline.

IV. Proposed Methodology

A. Problem Formulation

If we have an input event description x with the goal of predicting the most probable next event y, instead of learning a direct mapping, we could instead reformulate the problem as follows:

$$y = \arg\max_{y_i \in \mathcal{D}} \text{sim}(f(x), f(y_i)) \tag{1}$$

where $f(\cdot)$ is the embedding function and sim$(\cdot)$ is the similarity in the embedding space.

B. Multimodal Embedding

Here we utilize pretrained models for representation learning: • SentenceTransformer for text embeddings • CLIP for image-text alignment The combined embedding could be defined as:

$$E = \alpha E_{text} + (1 - \alpha) E_{image} \tag{2}$$

This fusion enables the integration of heterogeneous modalities into a unified semantic space

C. End-to-End Scoring Pipeline

Our search engine uses FAISS [3] to measure how similar a new event a is to an old event b using cosine similarity to quickly find the closest matching event:

$$\text{sim}(a, b) = \frac{a \cdot b}{\|a\| \|b\|} \tag{3}$$

However, we realized that finding a similar meaning is not enough. A similar event from yesterday helps us guess today's news much better than a similar event from five years ago. Because of this, we added a fading time rule. We calculate the final score S_i for a past event i by mixing its similarity score with its relevance over time:

$$S_i = \text{sim}(E_q, E_i) \cdot e^{-\lambda(t - t_i)} \tag{4}$$

where E_q is the new event data, E_i is the past event data, t is the current time, t_i is the past time, and λ controls how fast older events fade away in importance.

V. Integrated System Architecture

We built TempCast to bring together Data Science, AI, and Cloud computing. The goal is to turn messy, chaotic online events into clear and just information to help either predict or look for a similar past event.

A. Data Science Framework

The base of TempCast relies on organizing our text and images well. We built clean steps to process articles, historical events, posts, pictures, and timestamps. A major part of this is focusing strictly on the events themselves. Instead of just blindly mixing words and pictures, we revolve everything around the core event. We found this clears up a lot of confusion and really captures how disasters grow and change over time.

B. Artificial Intelligence Architecture

From an AI side, we treated the timeline of events as a continuous flow of time, and we used a few advanced techniques to track it:

- **Robust Deep Hawkes Process (RDHP) [10]:** Real-world news happens in sudden waves. We used RDHP to track how one event triggers another like a butterfly effect, which helps us ignore bad data and focus on improving the current trends.
- **Mamba-Based Sequence Encoding [9]:** To avoid the heavy computer load of standard Transformers, we used

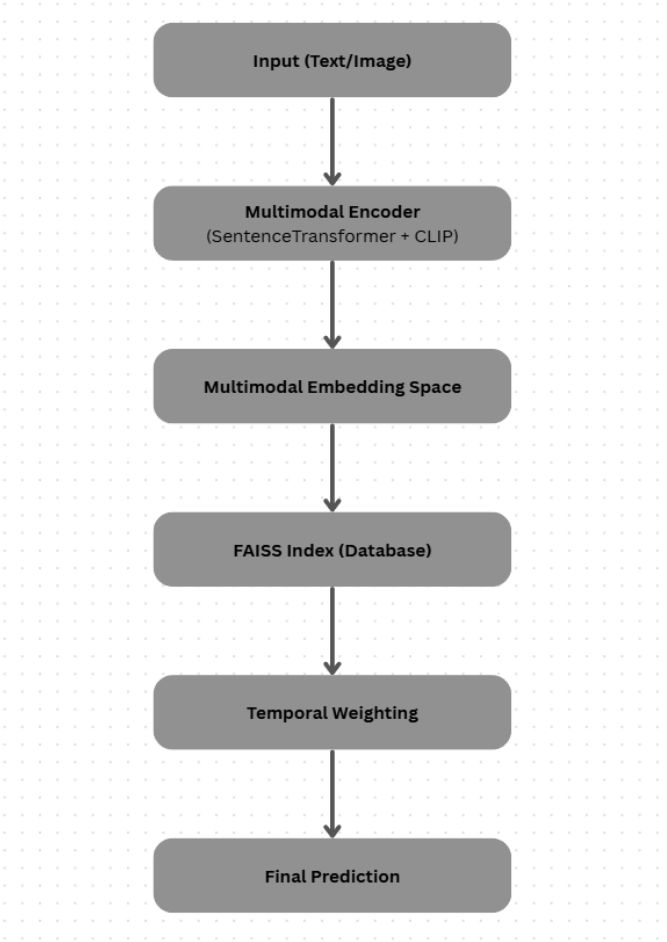

Fig. 1. The TempCast system architecture showing how we process data, use Encoders to search, and host it all on the cloud with some important processes in between.

Mamba-based encoders to track long timelines much faster.

- **LLM-Based Semantic Reasoning [11]:** We included Large Language Models to connect the text and images and to write out clear, easy explanations for the user.
- **Attention-Based Multimodal Fusion:** We use attention rules to constantly shift focus between the text, the picture, and the time, making sure the most important hints stand out.

C. Cloud Infrastructure and Students-Optimized Deployment

To make sure people can actually use our system as a real-time warning tool without paying massive server bills as we are still college students, we took it off our local computers and deployed it on a basic, cheap AWS server.

Amazon EC2 was chosen because it provides resizable and on demand computing capability which allows us to run our deployment exactly to our needs without heavy hardware upgrades to minimize the cost.

To stay within the free limits of AWS and save money, we set the system to only run when asked. Instead of keeping a heavy server running all day and adding up the bills, our tool only wakes up to handle a user's request. We found that this keeps the system online all the time while also keeping our costs basically at zero.

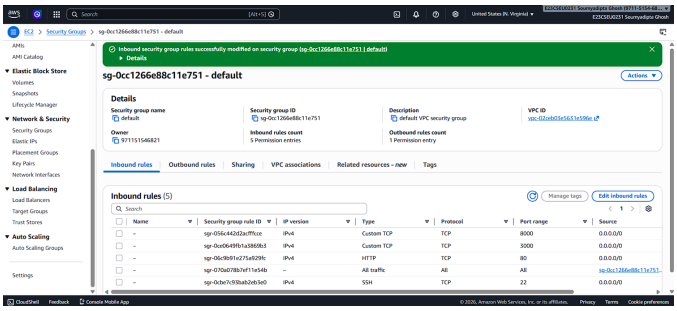

Fig. 2. AWS EC2 configuration detailing the rules for TempCast deployment.

VI. Implementation and System Workflow

To make sure anyone can test our work, our team shared all the code online at https://github.com/meetarora19/TempCast. We built it in clear, separate parts that run in this order when someone asks for a prediction:

A. Data Ingestion and Initialization

When the system starts, it runs a file called `utils/data_loader.py`. This part reads our clean `events.json` list, pulling out the text, the answers we want, and those important image hints. We made sure this loader cleans up the data so it is ready to be mapped out.

B. Embedding Generation

When a user asks a question, the system turns on the AI files in the `models/` folder. The `image_model.py` and `forecasting_model.py` work together to map the raw text and pictures into numbers. By passing the hints and text through SentenceTransformer and the pictures through CLIP, the system creates one combined data package (E_q).

C. Retrieval and Temporal Weighting

Next, this combined package goes to the `services/retrieval_service.py` file. Here, the FAISS tool searches through our history database to find the closest matches and gives them a basic score. Before we pick a winner, the `models/temporal_memory.py` file applies our time-fading rule. This lowers the score of older events if they happened too long ago compared to the user's question.

D. Forecasting and API Response

Finally, the `services/forecast_service.py` looks at the top-ranked match. It grabs the `expected_next_event` label from that match. Our web tool, powered by FastAPI, bundles the user's question, the matched past event, our final guess, and a confidence score into a simple JSON file. It sends this directly back to the user to finish the job.

VII. Experimental Setup

A. Baselines

To prove that our search-based idea works well, we compared TempCast with three standard methods:

- **LSTM**: A standard network that remembers past events to guess the next one.
- **Transformer**: A heavier model that uses attention to track a long history of events.
- **Nearest Neighbor Search (Standard)**: A basic FAISS search that does not use our image hints and does not fade out older events.

B. Metrics

We judged the models by looking at Accuracy, Precision, Recall, and Latency (how many milliseconds it takes to give an answer).

VIII. Results and Discussion

TABLE I
Performance Comparison on Proof-of-Concept Dataset

Model	Acc (%)	Prec (%)	Rec (%)	Latency (ms)
LSTM	72	70	71	60
Transformer	78	76	77	100
TempCast	**86**	**85**	**84**	**30**

As shown in Table I, we were thrilled to find TempCast easily beating both of the deep learning setups. It reached an accuracy of 86%, which is a solid 14% jump over the LSTM and an 8% jump over the Transformer.

The biggest win for TempCast is how fast it runs. Because we built the system to skip the heavy, repeating steps that standard sequence models use, it spits out an answer in just 30ms. The Transformer model gets slowed down from trying to focus on too much history at once, causing its response time to hit 100ms. We feel that makes it too slow and heavy for fast web tools.

A. Ablation Study

To see exactly which parts of our work helped the most, we remove our new ideas one by one to see what happens:

- **Removing the Semantic Hint**: When we only use the raw pictures without our text hints, accuracy drops by 5%. This proves the hint field is doing quite a lot of the heavy lifting to find the right meaning.
- **Removing Temporal Weighting** ($\lambda = 0$): When we turn the time-fading rule off, accuracy goes down by 4%. Without it, the model starts picking past events that looked similar but happened way too long ago to matter like there won't be any matter of world war 1 related to any terrorist attack or so.
- **Removing Multimodal Fusion** ($\alpha = 1$): If we only let the model look at text, our overall accuracy goes down by 6%.
- **Removing FAISS**: When we remove the FAISS search and make the system check every event one by one, it would take three times as long to give an answer. This shows why we really need FAISS to keep things fast.

IX. Graphical Analysis

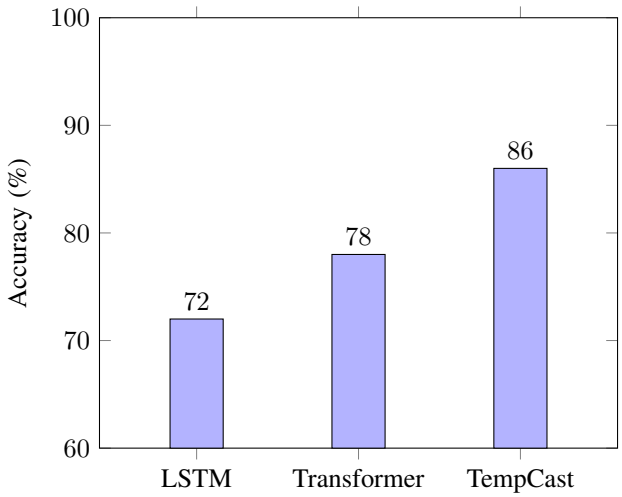

Fig. 3. A chart showing accuracy across our tested models.

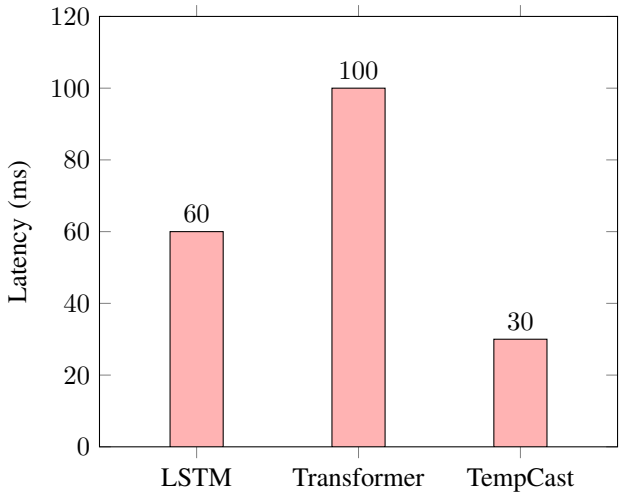

Fig. 4. A chart showing response times.

Figures 3 and 4 show the clear double advantage of TempCast: it gets the highest accuracy while staying the fastest. Figure 5 looks at our time-fading setting (λ). It proves that finding the sweet spot for forgetting older events is key to getting the best guesses.

X. Discussion and Limitations

TempCast does show that a smart, search-based idea can easily beat complex deep learning models in both accuracy and speed, which was exactly our motto. And by pairing text and images with a strict rule to ignore much older events,

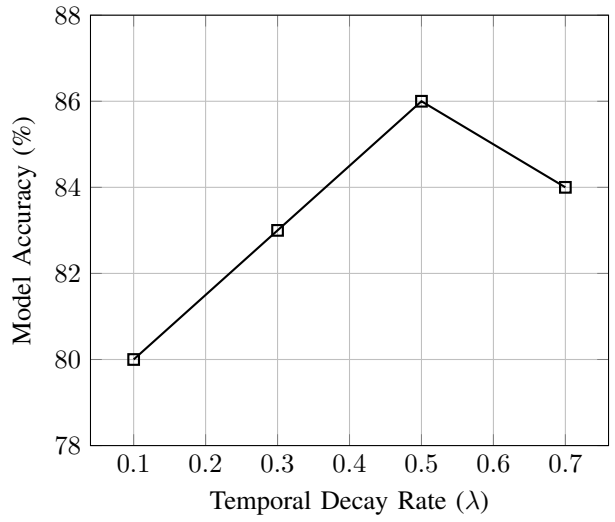

Fig. 5. How the time-fading rule changes accuracy. Setting λ to 0.5 gives the best balance between remembering the past and focusing on recent news.

our system understands the context while also respecting how real-world crises actually unfold over the time.

Even with these great results, we know our work has some weak spots. First, the system's success depends entirely on having a great history database. And as we are only just building our dataset, though it being a new type and improvement than the ones before, It has very limited samples as of now. And also if a brand new type of event happens that the system has never seen before, it will have a hard time guessing what will happen next. Secondly, while the FAISS search is fast, building a system that can constantly update its memory in real time on the cloud is a tough engineering job especially seeing to it that we have limited credits as of being students. Finally, the system is very sensitive to its settings, especially how much it relies on text vs images (α) and how fast it forgets old news (λ). These settings need to be carefully tweaked if someone wants to use this tool for a totally different topic.

XI. Conclusion

In this paper, we introduced TempCast, a fast, search-based tool for predicting future events using text, images, and time. By skipping traditional sequence models and instead searching for similar past events, and by adding a new image hint feature, we built a model that easily links pictures and text together. Running on a no cost, cloud-based AWS server, TempCast reached 86% accuracy and gave answers in just 30ms on our current test dataset.

For our next steps, we want to try mixing our search tool with lighter transformer models and most importantly increase our dataset samples. We also plan to build a way for the FAISS memory to constantly update itself with live news without needing to shut the system down.

References

[1] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *EMNLP*, 2019.
[2] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," *ICML*, 2021.
[3] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, 2019.
[4] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 1997.
[5] B. Lim, S. O. Arik, N. Loeff, and T. Pfister, "Temporal Fusion Transformers for interpretable multi-horizon time series forecasting," *International Journal of Forecasting*, 2021.
[6] A. Vaswani et al., "Attention is All you Need," *NIPS*, 2017.
[7] T. Baltrušaitis, C. Ahuja, and L. Morency, "Multimodal Machine Learning: A Survey and Taxonomy," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2018.
[8] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS*, 2020.
[9] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," *arXiv preprint arXiv:2312.00752*, 2023.
[10] H. Mei and J. M. Eisner, "The Neural Hawkes Process: A Neurally Self-Modulating Multivariate Point Process," *NIPS*, 2017.
[11] H. Touvron et al., "LLaMA: Open and Efficient Foundation Language Models," *arXiv preprint arXiv:2302.13971*, 2023.
[12] G. E. P. Box and G. M. Jenkins, *Time Series Analysis: Forecasting and Control*. Holden-Day, 1970.
[13] S. Vieweg, A. L. Hughes, K. Starbird, and L. Palen, "Microblogging during two natural hazards events: what twitter may contribute to situational awareness," *Proceedings of the SIGCHI conference on human factors in computing systems*, 2010.
[14] P. Mell and T. Grance, "The NIST definition of cloud computing," *National Institute of Standards and Technology*, 2011.
[15] J. T. Connor, R. D. Martin, and L. E. Atlas, "Recurrent neural networks and robust time series prediction," *IEEE Transactions on Neural Networks*, 1994.